

MSC IN MATHEMATICAL ENGINEERING

COURSE: NUMERICAL ANALYSIS FOR PARTIAL DIFFERENTIAL EQUATIONS

Lecturer: Prof Paola F. Antonietti
Teaching Assistant: Prof. F. Regazzoni
Tutor: Dr. N. De March

Type B project
A.Y. 2025/2026

Stochastic Iterative Solvers for Complex Fluid Flows Problems on Polygonal Meshes

Authors:

Christian Capelli
Matteo Morini

Tutors:

Paola Francesca Antonietti
Gabriele Ciaramella
Alessandra Cancrini

Abstract

This work focuses on the development of stochastic iterative solvers for the pseudo-stress formulation of the unsteady Stokes problem, discretised by means of a discontinuous Galerkin method on polytopal grids (PolyDG) and a Crank-Nicolson time scheme. At each time step, we solve the resulting linear systems with a flexible AB-GMRES method preconditioned by Kaczmarz-type inner iterations. We propose and compare greedy, randomized, and greedy-randomized variants against the standard NE-SOR approach.

Numerical tests on well-conditioned and ill-conditioned matrices show that greedy methods outperform randomized and classical variants, particularly when more inner work is required. However, on the PolyDG discretization, inner iterations stagnate due to the block structure of the system matrix. We address this by proposing an adaptive AB-GMRES approach that adjusts the number of inner iterations based on residual behavior to avoid wasted iterations on stagnated sweeps.

Contents

Abstract	1
1 Introduction	3
2 Flexible AB-GMRES with Kaczmarz-Type Inner Iterations	4
2.1 The GMRES method	4
2.2 The Kaczmarz method	5
2.3 The algorithm	6
2.4 Tests and results	8
3 Model problem and numerical discretization	12
3.1 Pseudo-stress weak formulation.	12
3.2 PolyDG semi-discrete formulation.	12
3.3 PolyDG fully-discrete formulation	14
4 Numerical Results	15
4.1 Theoretical Error Bounds and Expected Observations	15
4.2 Solver Performance and Inner Iteration Stagnation	16
4.3 Adaptive AB-GMRES method	20
5 Conclusion	23
Bibliography	24

1 Introduction

The numerical approximation of incompressible viscous flows is a central problem in computational fluid dynamics. In this work, we consider the pseudo-stress formulation of the unsteady Stokes equations and discretise it with a discontinuous Galerkin method on polytopal meshes (PolyDG). For the time integration, we use a Crank–Nicolson scheme. At each time step, this leads to a linear system with a symmetric positive definite matrix. For the formulation of the problem and the PolyDG discretisation, we refer to [1].

Our main goal is to construct iterative solvers with stochastic preconditioners based on the Kaczmarz method [6] to solve the Unsteady Stokes problem in its pseudostress formulation. A standard approach is the right-preconditioned GMRES method, also known as AB-GMRES, introduced by Hayami, Yin and Ito in [7]. In this framework, the preconditioner is applied through inner iterations rather than by forming a preconditioning matrix explicitly. This reduces storage requirements and can improve convergence.

A common choice for the inner solver in AB-GMRES is the NE-SOR method, also known as the Kaczmarz or row-action method. In recent years, more effective Kaczmarz-type methods have been proposed, including greedy, randomized, and greedy randomized variants. In this work, we replace NE-SOR with these Kaczmarz-type inner iterations and compare their behaviour.

When Kaczmarz-type inner iterations are used, the inner preconditioner changes from one iteration to the next. For this reason, the resulting method is a flexible AB-GMRES scheme, with flexible GMRES used as the outer iteration and Kaczmarz-type methods used as inner iterations. For more in depth details, we refer to [6].

The report is organised as follows. Section 2 introduces the flexible AB-GMRES method with Kaczmarz-type inner iterations and discusses the implemented variants. Section 3 presents the model problem with the space and time discretisations. Section 4 reports the complete numerical results and discusses the performance of the proposed solver, introducing an adaptive version of the solver.

2 Flexible AB-GMRES with Kaczmarz-Type Inner Iterations

We start by considering the problem of solving a consistent linear system

$$A\mathbf{x} = \mathbf{b}, \quad \mathbf{b} \in \mathcal{R}(A),$$

where $A \in \mathbb{R}^{m \times n}$ is not necessarily square or full rank, and $\mathcal{R}(A)$ denotes the range space of A . In this setting, one is often interested in the minimum Euclidean-norm solution,

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{x}\|_2 \quad \text{s.t.} \quad A\mathbf{x} = \mathbf{b}. \quad (1)$$

A standard approach is to rewrite the problem through the normal equations

$$AA^\top \mathbf{u} = \mathbf{b}, \quad \mathbf{x} = A^\top \mathbf{u}. \quad (2)$$

For large and sparse matrices, direct methods are usually too computationally expensive, so iterative methods are preferred.

2.1 The GMRES method

GMRES is a classical Krylov subspace method for solving nonsymmetric linear systems by minimizing the residual norm over an expanding Krylov subspace. For more details on the method, its implementation aspects and its convergence theory, we refer to [5].

A preconditioned variant in this setting is the AB-GMRES method, proposed by Hayami, Yin and Ito in [7] which applies GMRES to a right-preconditioned formulation of the problem. The idea is to apply GMRES to $\min_{\mathbf{u} \in \mathbb{R}^m} \|\mathbf{b} - AB\mathbf{u}\|_2$, where $B \in \mathbb{R}^{n \times m}$ is a right preconditioner.

In particular, instead of explicitly applying preconditioning matrices, we apply inner iterations as a preconditioner inside the Krylov subspace methods in order to save the storage requirement for the preconditioning matrix and possibly to accelerate the convergence. Such techniques are often called inner-outer iteration methods.

Algorithm 1 AB-GMRES method preconditioned by inner iterations [6]

- 1: Let $\mathbf{x}_0$ be the initial approximate solution and $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$.
 - 2: $\beta = \|\mathbf{r}_0\|_2$, $\mathbf{v}_1 = \mathbf{r}_0/\beta$
 - 3: **for** $k = 1, 2, \dots$ until convergence **do**
 - 4: Apply ℓ iterations of a stationary iterative method to $AA^\top \mathbf{y} = \mathbf{v}_k$, $\mathbf{z} = A^\top \mathbf{y}$ to obtain $\mathbf{z}_k = B^{(\ell)} \mathbf{v}_k$.
 - 5: $\mathbf{w}_k = A\mathbf{z}_k$
 - 6: **for** $i = 1, 2, \dots, k$, **do**
 - 7: $h_{i,k} = \mathbf{w}_k^\top \mathbf{v}_i$, $\mathbf{w}_k = \mathbf{w}_k - h_{i,k} \mathbf{v}_i$
 - 8: **end for**
 - 9: $h_{k+1,k} = \|\mathbf{w}_k\|_2$, $\mathbf{v}_{k+1} = \mathbf{w}_k/h_{k+1,k}$
 - 10: **end for**
 - 11: $\mathbf{y}_k = \arg \min_{\mathbf{y} \in \mathbb{R}^k} \|\beta \mathbf{e}_1 - \bar{H}_k \mathbf{y}\|_2$, $\mathbf{u}_k = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k] \mathbf{y}_k$, where $\bar{H}_k = \{h_{ij}\}_{1 \leq i \leq k+1; 1 \leq j \leq k}$ and $\mathbf{e}_1 = [1, 0, \dots, 0]^\top$
 - 12: Apply ℓ iterations of a stationary iterative method to $AA^\top \mathbf{y} = \mathbf{u}_k$, $\mathbf{z} = A^\top \mathbf{y}$ to obtain $\mathbf{z}_k = B^{(\ell)} \mathbf{u}_k$.
 - 13: $\mathbf{x}_k = \mathbf{x}_0 + \mathbf{z}_k$
-

In the AB-GMRES preconditioned by inner iterations, one common choice for stationary inner iterations is the NE-SOR method, which is mathematically equivalent to the SOR method applied to the normal equations of the second kind [4]. More specifically, if we use $\boldsymbol{\alpha}_i^\top$ to represent the i th row of the matrix A , and v_i to represent the i th entry of the vector $\mathbf{v}$, then the NE-SOR method for $AA^\top \mathbf{y} = \mathbf{v}$, $\mathbf{z} = A^\top \mathbf{y}$ can be described as follows.

Algorithm 2 NE-SOR method [8]

```

1: Let  $\mathbf{z}^{(0)}$  be the initial approximate solution and  $\omega \in \mathbb{R}$  be the relaxation parameter.
2: for  $p = 0, 1, 2, \dots, \ell - 1$  do
3:   for  $i = 1, 2, \dots, m$  do
4:      $\mathbf{z}^{(p)} = \mathbf{z}^{(p)} + \omega \frac{v_i - \boldsymbol{\alpha}_i^\top \mathbf{z}^{(p)}}{\|\boldsymbol{\alpha}_i\|_2^2} \boldsymbol{\alpha}_i$ 
5:   end for
6:    $\mathbf{z}^{(p+1)} = \mathbf{z}^{(p)}$ 
7: end for

```

2.2 The Kaczmarz method

In this report, we focus on Kaczmarz-type inner iterations. The Kaczmarz method is a row-action method with a simple geometric interpretation: each step projects the current iterate orthogonally onto the hyperplane defined by one equation of the system. Let $A \in \mathbb{R}^{m \times n}$ have rows $\boldsymbol{\alpha}_1^\top, \dots, \boldsymbol{\alpha}_m^\top$, and let $\mathbf{v} \in \mathbb{R}^m$ denote the right-hand side of the inner system. If the i -th row of A is denoted by $\boldsymbol{\alpha}_i^\top$ and the corresponding component of $\mathbf{v}$ by v_i , then the hyperplane associated with that equation is $\boldsymbol{\alpha}_i^\top \mathbf{z} = v_i$. Starting from an initial guess $\mathbf{z}^{(0)}$, the iterate is updated by projecting it onto one hyperplane at a time. The generic Kaczmarz correction at the p -th inner iteration is

$$\mathbf{z}^{(p+1)} = \mathbf{z}^{(p)} + \omega \frac{v_i - \boldsymbol{\alpha}_i^\top \mathbf{z}^{(p)}}{\|\boldsymbol{\alpha}_i\|_2^2} \boldsymbol{\alpha}_i, \quad (3)$$

where ω is the relaxation parameter. When the rows are visited cyclically, one full sweep through all rows is completed in m inner steps.

Several Kaczmarz-type variants have been proposed according to the rule used to select the working row at each inner iteration. In all cases, the correction formula remains the same; only the choice of the index i_p changes. We will briefly describe each of them:

i. Classical Kaczmarz method. The classical Kaczmarz method visits the rows of A cyclically. In iteration p , the working row is chosen as $i_p = (p \bmod m) + 1$.

ii. Greedy Kaczmarz method. The greedy Kaczmarz method selects, at each inner iteration, the row whose current residual component has the largest absolute value:

$$i_p = \arg \max_{1 \leq i \leq m} |v_i - \boldsymbol{\alpha}_i^\top \mathbf{z}^{(p)}|.$$

In this way, the method always acts on the equation that is currently least satisfied. The idea is to reduce the largest error component first, which often leads to faster convergence in practice.

iii. Randomized Kaczmarz method. The randomized Kaczmarz method randomly chooses the working row according to a probability distribution. In the version used here, the row i is selected with probability

$$\Pr(\text{row} = i) = \frac{\|\boldsymbol{\alpha}_i\|_2^2}{\|A\|_F^2},$$

where $\|A\|_F$ is the Frobenius norm of A . Thus, rows with larger Euclidean norm are selected more frequently. Once the row has been chosen, the same projection formula is applied.

iv. Greedy randomized Kaczmarz method. The greedy randomized Kaczmarz method combines the ideas of the previous two variants. First it identifies a subset of rows whose residual contributions are greater than a given threshold. Only those rows will be updated. Then the method randomly chooses among that subset of rows with the same probability criteria as the randomized variant.

Remark 1. For greedy methods, $\mathbf{s} = \mathbf{v} - A\mathbf{z}$ needs to be calculated at each inner iteration. This additional computational work cannot be ignored. On the other hand, we may update the residual vector $\mathbf{s}$ recursively as follows [3]:

$$\begin{aligned}
\mathbf{s}^{(p+1)} &= \mathbf{v} - A\mathbf{z}^{(p+1)} \\
&= \mathbf{v} - A \left(\mathbf{z}^{(p)} + \omega \frac{v_{i_p} - \boldsymbol{\alpha}_{i_p}^\top \mathbf{z}^{(p)}}{\|\boldsymbol{\alpha}_{i_p}\|_2^2} \boldsymbol{\alpha}_{i_p} \right) \\
&= \mathbf{v} - A\mathbf{z}^{(p)} - \omega \frac{s_{i_p}^{(p)}}{\|\boldsymbol{\alpha}_{i_p}\|_2^2} A\boldsymbol{\alpha}_{i_p} \\
&= \mathbf{s}^{(p)} - \omega \frac{s_{i_p}^{(p)}}{\|\boldsymbol{\alpha}_{i_p}\|_2^2} C_{(i_p)}. \tag{4}
\end{aligned}$$

Here, $C_{(i_p)}$ is the i_p th column of $C = AA^\top$. Hence, if the matrix product $AA^\top$ is computed and stored once in the beginning, the computational work can be reduced, assuming that the total number of inner iterations is more than the number of rows of A .

We then replace the explicit preconditioning matrix B in the AB-GMRES method by a finite number of inner Kaczmarz iterations.

The number of inner iterations does not have to be fixed in advance and may vary from one outer iteration to the next. This flexibility is one of the main advantages of the method, since the amount of inner work can be adapted dynamically: too few inner iterations may produce a weak preconditioner but too many may increase the computational cost without sufficient benefit.

2.3 The algorithm

After implementing the GMRES and AB-GMRES algorithms, here we present only the final version of the flexible AB-GMRES method with Kaczmarz-type inner iterations. For the intermediate variants, we refer to [6].

The algorithm starts from the normalized initial residual $\mathbf{v}_1 = \frac{\mathbf{b} - A\mathbf{x}_0}{\|\mathbf{r}_0\|_2}$. This vector is the first element of the orthonormal basis $V_k = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k]$, which represents the Krylov space built by the outer flexible GMRES iteration: $\mathbf{v}_1$ gives the initial direction from which the residual space is expanded. More on V_k will be explained later.

The (flexible) preconditioner works as follows: during each outer step, we apply a Kaczmarz-type inner iteration to the current vector $\mathbf{v}_k$. Since the number of inner iterations is no longer fixed in advance, it is now determined by the stopping criterion controlled by the parameter $\eta \in (0, 1)$. More precisely, the inner process is stopped as soon as the relative residual of the inner problem satisfies $\|\mathbf{v}_k - AB^{(\ell)}\mathbf{v}_k\|_2 \leq \eta\|\mathbf{v}_k\|_2$. A smaller value of η gives a more accurate inner solve, while a larger value reduces the inner work but may lead to a weaker preconditioner. The result is a vector $\mathbf{z}_k$, which plays the role of a preconditioned search direction. All such vectors are collected in the matrix

$$Z_k = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k] \in \mathbb{R}^{n \times k}.$$

This matrix stores the directions produced by the inner solver and will be used later, outside the loop, to reconstruct the approximate solution in the end.

Algorithm 3 Flexible AB-GMRES preconditioned by Kaczmarz-type methods [6]

```

1: Let  $\mathbf{x}_0$  be the initial approximate solution and  $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ .
2:  $\beta = \|\mathbf{r}_0\|_2$ ,  $\mathbf{v}_1 = \mathbf{r}_0/\beta$ 
3: for  $k = 1, 2, \dots$  until convergence do
4:   Apply  $\ell_k$  iterations of a Kaczmarz-type method to  $A\mathbf{z} = \mathbf{v}_k$  to obtain  $\mathbf{z}_k = B^{(\ell_k)}\mathbf{v}_k$ ,
     where  $\ell_k = \min\{\ell_{\max}, \ell\}$ , and  $\ell_{\max}$  is the maximum number of inner iterations allowed, and
      $\ell$  is the smallest  $\ell$  such that

$$\|\mathbf{v}_k - AB^{(\ell)}\mathbf{v}_k\|_2 \leq \eta \|\mathbf{v}_k\|_2.$$

5:    $\mathbf{w}_k = A\mathbf{z}_k$ 
6:   for  $i = 1, 2, \dots, k$  do
7:      $h_{i,k} = \mathbf{w}_k^\top \mathbf{v}_i$ ,  $\mathbf{w}_k = \mathbf{w}_k - h_{i,k}\mathbf{v}_i$ 
8:   end for
9:    $h_{k+1,k} = \|\mathbf{w}_k\|_2$ ,  $\mathbf{v}_{k+1} = \mathbf{w}_k/h_{k+1,k}$ 
10: end for
11:  $\mathbf{y}_k = \arg \min_{\mathbf{y} \in \mathbb{R}^k} \|\beta \mathbf{e}_1 - \bar{H}_k \mathbf{y}\|_2$ ,  $\mathbf{u}_k = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k] \mathbf{y}_k$ , where  $\bar{H}_k = \{h_{ij}\}_{1 \leq i \leq k+1; 1 \leq j \leq k}$ 
12:  $\mathbf{x}_k = \mathbf{x}_0 + \mathbf{u}_k$ 

```

At this point we compute $\mathbf{w}_k = A\mathbf{z}_k$ and orthogonalize it against all the previously computed basis vectors up to now $\mathbf{v}_1, \dots, \mathbf{v}_k$ using a Gram–Schmidt process. Thanks to the orthogonalization we remove all previously existing directions from $\mathbf{w}_k$ and we are left with a brand new direction that expands our Krylov space. We normalize it and append it in V_k . It is now clear that the matrix V_k starting from $\mathbf{v}_1 = \mathbf{r}_0/\|\mathbf{r}_0\|_2$ collects the orthonormal residual vectors generated by each outer iteration and forms a growing orthonormal basis of the Krylov subspace.

The corresponding projection coefficients of each $\mathbf{w}_k$ into each $\mathbf{v}_i$, namely $h_{i,k}$, are stored in the matrix $\bar{H}_k \in \mathbb{R}^{(k+1) \times k}$, which is an upper Hessenberg matrix. Its entries $h_{i,k}$ measure how much of $\mathbf{w}_k$ already lies in the old directions $\mathbf{v}_i$, while the final coefficient $h_{k+1,k} = \|\mathbf{w}_k\|_2$ measures the size of the new component that remains after it is stripped of all pre-existing directions.

When the vector $\mathbf{w}_k$ has no component orthogonal to the already constructed basis, no new basis vector $\mathbf{v}_{k+1}$ can be generated. This means that the current Krylov subspace has reached its maximal dimension for the given process. This situation is reflected by the fact that $\|\mathbf{w}_k\|_2 = 0$, so that $h_{k+1,k} = 0$, which in turn leads to a division by zero in line 9 of the algorithm and corresponds to a breakdown of the orthogonalization process. See remark 2.

$$\bar{H}_k = \begin{bmatrix} h_{11} & h_{12} & h_{13} & \dots \\ h_{21} & h_{22} & h_{23} & \dots \\ 0 & h_{32} & h_{33} & \dots \\ \vdots & 0 & h_{43} & \dots \\ \vdots & \vdots & \ddots & \ddots \end{bmatrix} \quad (5)$$

The Hessenberg matrix $\bar{H}_k$. For each k , the coefficients $h_{i,k} = \mathbf{w}_k^\top \mathbf{v}_i$ for $i = 1, \dots, k$ represent the projections of the current vector $\mathbf{w}_k$ onto the existing basis vectors $\mathbf{v}_i$. The coefficient $h_{k+1,k} = \|\mathbf{w}_k\|_2$ (the one below the diagonal) is the normalization factor for the next basis vector.

Finally, we obtain the coefficients $\mathbf{y}_k$ solving $\mathbf{y}_k = \arg \min_{\mathbf{y} \in \mathbb{R}^k} \|\beta \mathbf{e}_1 - \bar{H}_k \mathbf{y}\|_2$, where $\beta = \|\mathbf{r}_0\|_2$. This problem determines the optimal weights used to combine the preconditioned directions stored in Z_k . Then we can map the resulting linear combination back to the solution space and update the solution: $\mathbf{x}_k = \mathbf{x}_0 + Z_k \mathbf{y}_k$.

Remark 2 (Fatal breakdown). *When the number of inner iterations is too low, or when η is not sufficiently small in the adaptive stopping criterion, the method may break down. This happens because the inner Kaczmarz process cannot provide a sufficiently effective approximation of the*

*preconditioner in too little steps, so the new Krylov basis vector is no longer linear independent from the previous ones, the Krylov space expansion stagnates and cannot be continued. This corresponds to the Hessenberg matrix (5) becoming singular*¹.

Parameter Tuning. Algorithm 3 involves two parameters that control the inner Kaczmarz-type preconditioner: the maximum number of inner iterations $\ell_{\max}$ and the relaxation parameter ω . Since the computational cost of the solver depends on these parameters, it is useful to choose them automatically rather than by hand. We implemented a `parameter_tuning` routine to address this problem and adaptively select the parameters at each time step.

Algorithm 4 Parameter tuning procedure of Kaczmarz-type methods [6]

- 1: Set $\omega = 1$ and $\mathbf{z}^{(0)} = 0$.
- 2: Apply a Kaczmarz-type method to $A\mathbf{z} = \mathbf{b}$ until the ℓ th iteration $\mathbf{z}^{(\ell)}$ satisfies

$$\|\mathbf{b} - A\mathbf{z}^{(\ell)}\|_2 \leq \eta \|\mathbf{b}\|_2.$$

- 3: $\ell_{\max} = \ell$
 - 4: **for** $\omega = 0.1, 0.2, \dots, 1.9$ **do**
 - 5: Apply $\ell_{\max}$ iterations of a Kaczmarz-type method to $A\mathbf{z} = \mathbf{b}$.
 - 6: **end for**
 - 7: $\omega_{\text{opt}} = \arg \min_{\omega=0.1, 0.2, \dots, 1.9} \|\mathbf{b} - A\mathbf{z}^{(\ell_{\max})}\|_2$
-

2.4 Tests and results

We first attempted to reproduce the experiments reported in [1] in order to verify the consistency of our MATLAB implementation with the expected numerical behavior. In the following tests we have always used the Parameter Tuning function introduced in 2.3 to find the optimal ω and the maximum number of inner iterations possible.

Table 1. Matrices used in the tests.

Type	Name	m	n	Density[%]	Rank	$\kappa(A)$
Underdetermined	RANDL1	500	5000	20	500	1.00×10^1
	RANDL6	500	5000	20	500	1.00×10^6
Overdetermined	RANDL1	5000	500	20	500	1.00×10^1
	RANDL6	5000	500	20	500	1.00×10^6

Undetermined problems. We begin with the full-rank matrix `RANDL1` of size 500×5000 , using `tol` = 10^{-6} and three different values of η , namely 10^{-1} , 10^{-2} , and 10^{-3} . When tested with $\eta = 10^{-1}$ the choice of inner preconditioner has less room to make a dramatic difference and the convergence rate of all methods looks very similar. On the other hand, with smaller η more inner work is required (see algorithm 3), and greedy methods seem to gain a slight advantage (in terms of outer iterations required) over randomized and regular methods. This is a behavior that will be even more pronounced for more ill-conditioned matrices.

We then tested another matrix `RANDL6` of the same size and density, but with a much larger condition number, $\kappa(A) \approx 10^6$ (Again using the same tolerance `tol` = 10^{-6}).

¹In `GMRES`, the case where H becomes singular has a different interpretation and corresponds to a lucky breakdown, meaning that the approximate solution coincides with the exact one. See [5, 6].

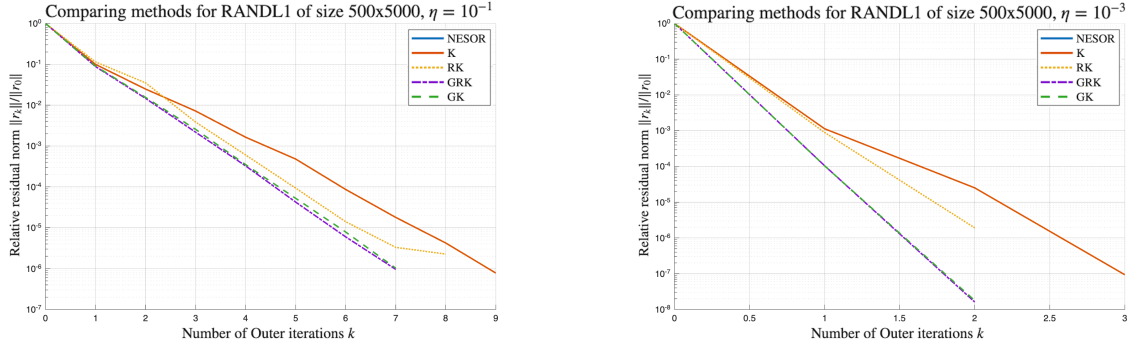


Fig. 1. Convergence behavior of the matrix RANDL1 for $\eta = 10^{-1}$ (left) and $\eta = 10^{-3}$ (right). Notice how more inner work leads to a smaller number of outer iterations, and how the greedy methods gain a certain advantage as more inner work is required.

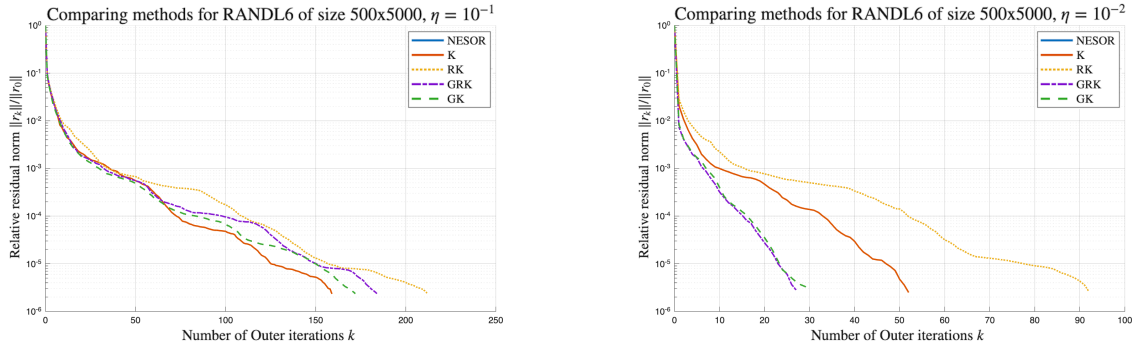


Fig. 2. Convergence behavior of the matrix RANDL6 for $\eta = 10^{-1}$ (left) and $\eta = 10^{-2}$ (right). For $\eta = 10^{-1}$ GR and GRK start to show an increased performance. For $\eta = 10^{-2}$ GK and GRK are definitely the most effective variants in terms of how many outer iterations are required, but the results in table 2 show how GRK pays for this with larger inner cost.

The higher conditioning number leads to an increased number of outer iterations performed. We also note the same dependence on η . Smaller values of η force more inner work, leading to a more accurate (although more computationally expensive) preconditioner. In our experiments, GRK and GK seem to benefit the most from this extra inner accuracy. In fact, since GK and GRK are row-selection strategies that adapt to the current residual, they can exploit additional inner iterations more effectively when the inner solver is allowed to run longer, while on the other hand the classical Kaczmarz and NE-SOR variants are more structured and may already be near their practical limit for this matrix and parameter regime, so decreasing η does not change their outer behavior as much. We collect all data from the tests on RANDL6 in Table 2 to make further considerations on the CPU time of all variants.

Overdetermined problems. We now replicate the same exact tests for overdetermined problems (The test matrices are listed in Table 1):

Table 2. Results for the underdetermined RANDL6. AB-GMRES with NE-SOR and F-AB-GMRES with classical Kaczmarz remain very close, with identical outer iteration counts. The randomized variants require much more inner work, especially RK, which is the most expensive in all three cases. GK is the fastest method, while GRK improves in outer iterations as η decreases, but pays for this with larger inner cost.

Outer iteration	Inner iteration	$\eta = 10^{-1}$	$\eta = 10^{-2}$	$\eta = 10^{-3}$
AB-GMRES	NE-SOR	159 (400000, 2516, 1.0) 97.34 (0.18) 9.73×10^{-7}	52 (1351500, 25990, 0.8) 331.82 (1.82) 8.66×10^{-7}	59 (1530000, 25932, 0.8) 651.76 (1.71) 8.65×10^{-7}
F-AB-GMRES	K	159 (397500, 2500, 1.0) 105.59 (0.18) 9.73×10^{-7}	52 (1326000, 25500, 0.8) 347.35 (1.76) 8.66×10^{-7}	59 (1504500, 25500, 0.8) 393.32 (1.74) 8.65×10^{-7}
	RK	212 (945732, 4461, 1.1) 266.64 (7.17) 9.82×10^{-7}	92 (2300092, 25001, 1.2) 740.56 (39.82) 9.14×10^{-7}	93 (2325093, 25001, 1.1) 709.30 (39.30) 9.05×10^{-7}
	GRK	184 (138552, 753, 1.1) 44.94 (1.74) 9.82×10^{-7}	27 (473310, 17530, 1.4) 174.23 (40.79) 9.78×10^{-7}	25 (625025, 25001, 1.4) 230.49 (53.71) 8.63×10^{-7}
	GK	172 (142416, 828, 1.2) 42.91 (0.12) 9.83×10^{-7}	30 (506700, 16890, 1.4) 146.60 (1.79) 9.58×10^{-7}	27 (675027, 25001, 1.3) 185.64 (2.39) 9.12×10^{-7}

First row: Number of outer iterations (total number of inner iterations, average number of inner iterations per outer iteration, ω).

Second row: Total CPU time, which includes parameter tuning time in parentheses, in seconds.

Third row: Relative residual norm.

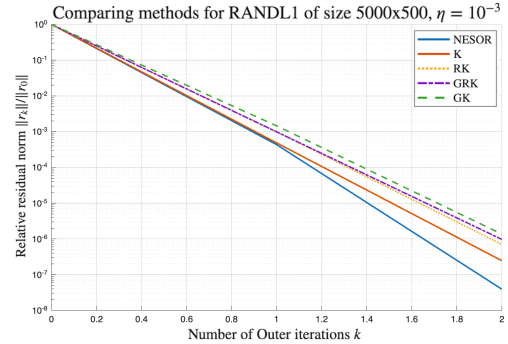
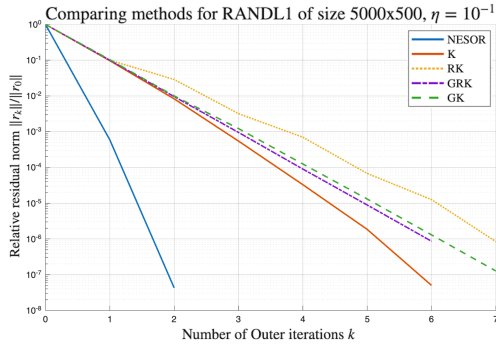


Fig. 3. Convergence behavior of the overdetermined matrix RANDL1 for $\eta = 10^{-1}$ and $\eta = 10^{-3}$.

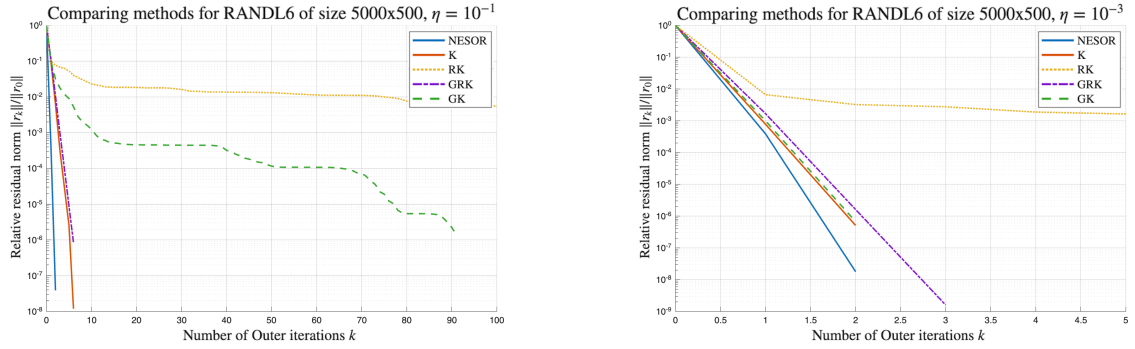


Fig. 4. Convergence behavior of the overdetermined matrix RANDL6 for $\eta = 10^{-1}$ and $\eta = 10^{-3}$. The slow converging curve of RK has been excluded for visual reasons. But the complete results are displayed in table 3

Table 3. Results for the overdetermined RANDL6. AB-GMRES with NE-SOR and F-AB-GMRES with classical Kaczmarz remain close in outer iterations and CPU time. GK is again the fastest method overall. RK is again the most expensive in terms of inner work, and this gap becomes even more pronounced as the number of inner iterations grows for smaller values of η .

Outer iteration	Inner iteration	$\eta = 10^{-1}$			$\eta = 10^{-2}$			$\eta = 10^{-3}$		
AB-GMRES	NE-SOR	2	(15000, 7500, 0.9)		2	(15000, 7500, 1.0)		2	(15000, 7500, 1.0)	
		2.31	(1.62)		2.86	(1.97)		3.72	(2.89)	
		2.66×10^{-8}	2.91×10^{-8}		2.51×10^{-8}	2.84×10^{-8}		1.22×10^{-8}	1.65×10^{-8}	
F-AB-GMRES	K	6	(17387, 2898, 0.9)		3	(9969, 3323, 1.0)		2	(8332, 4166, 1.0)	
		4.53	(1.20)		4.36	(1.88)		5.15	(3.51)	
		8.16×10^{-9}	1.01×10^{-8}		5.58×10^{-7}	1.31×10^{-6}		3.46×10^{-7}	1.14×10^{-6}	
	RK	310	(1715051, 5532, 0.9)		77	(7343781, 95439, 1.2)		48	(12000048, 250001, 1)	
		331.20	(81.67)		2028.81	(1306.05)		4524.14	(3422.74)	
		4.89×10^{-7}	2.24×10^{-2}		5.94×10^{-7}	5.96×10^{-3}		2.75×10^{-7}	1.04×10^{-2}	
	GRK	6	(1864, 311, 1.0)		3	(1502, 501, 1.0)		3	(1977, 659, 1.0)	
		8.51	(6.19)		13.07	(11.25)		15.02	(13.22)	
		5.68×10^{-7}	7.21×10^{-7}		7.39×10^{-7}	7.46×10^{-7}		1.08×10^{-9}	1.43×10^{-9}	
	GK	91	(32079, 353, 1.0)		5	(7305, 1461, 0.7)		2	(5740, 2870, 0.7)	
		28.13	(0.51)		3.67	(1.31)		4.12	(2.42)	
		9.50×10^{-7}	2.98×10^{-3}		2.70×10^{-8}	1.60×10^{-4}		4.58×10^{-7}	3.62×10^{-3}	

First row: Number of outer iterations (total number of inner iterations, average number of inner iterations per outer iteration, ω).

Second row: Total CPU time, which includes parameter tuning time in parentheses, in seconds.

Third row: Relative residual norm and relative error norm.

3 Model problem and numerical discretization

3.1 Pseudo-stress weak formulation.

We focus on the Stokes problem, which models incompressible viscous free flows, and formulate it in terms of a pseudo-stress unknown rather than its classical expression. The derivation of the pseudo-stress formulation for the unsteady Stokes problem is presented in [1]. The pseudo-stress variable is defined as

$$\boldsymbol{\sigma}(\mathbf{u}, p) = \mu \nabla \mathbf{u} - p \mathbb{I}_d,$$

where $\mathbf{u}$ is the flow velocity and p its pressure. Let $\Omega \subset \mathbb{R}^d$, $d = 2$, be an open, convex polygonal domain with Lipschitz boundary, and let

$$\text{dev}(\boldsymbol{\tau}) = \boldsymbol{\tau} - \frac{1}{d} \text{tr}(\boldsymbol{\tau}) \mathbb{I}_d$$

be the deviatoric operator, where $\text{tr}(\cdot)$ is the trace operator. Then, the Stokes problem can be rewritten as

$$\begin{cases} \frac{1}{\mu} \frac{\partial \text{dev}(\boldsymbol{\sigma})}{\partial t} - \nabla \cdot (\nabla \cdot \boldsymbol{\sigma}) = \mathbf{F} & \text{in } \Omega \times (0, T], \\ \mathbf{u} = \mathbf{g}_D & \text{on } \Gamma_D \times (0, T], \\ \boldsymbol{\sigma} \mathbf{n} = \mathbf{g}_N & \text{on } \Gamma_N \times (0, T], \\ \text{dev}(\boldsymbol{\sigma})(\cdot, t = 0) = \boldsymbol{\sigma}_0 & \text{in } \Omega, \end{cases} \quad (6)$$

where $\mu > 0$ is the fluid viscosity, and $T > 0$ the final simulation time. The boundary of Ω is partitioned as $\Gamma_D \cup \Gamma_N = \partial\Omega$, with $\Gamma_D \cap \Gamma_N = \emptyset$. For simplicity, we assume both $|\Gamma_D| > 0$ and $|\Gamma_N| > 0$. We also assume that $\mathbf{F} \in \mathbf{H}^1(\Omega)$, $\mathbf{g}_N \in \mathbf{L}^2(\Gamma_N)$, $\boldsymbol{\sigma}_0 \in \mathbf{L}^2(\Omega)$ and that $\forall t$ the Dirichlet datum $\mathbf{g}_D = \mathbf{g}_D(t)$ is the trace of a function in $\mathbf{H}^1(\Omega)$. To strongly enforce the essential traction condition on Γ_N we define

$$\mathbb{H}_{0,\Gamma_N}(\text{div}, \Omega) = \{\boldsymbol{\eta} \in \mathbf{H}(\text{div}, \Omega) \mid \langle \boldsymbol{\eta} \mathbf{n}, \mathbf{v} \rangle_{\partial\Omega} = 0 \quad \forall \mathbf{v} \in \mathbf{H}_{0,\Gamma_D}^1(\Omega)\}$$

where

$$\mathbf{H}_{0,\Gamma_D}^1(\Omega) = \{\mathbf{v} \in \mathbf{H}^1(\Omega)^d \mid \mathbf{v} = \mathbf{0} \text{ on } \Gamma_D\}.$$

Then, the corresponding weak formulation reads as: for any $t \in (0, T]$, find $\boldsymbol{\sigma}(t) \in \mathbb{H}_{0,\Gamma_N}(\text{div}, \Omega)$ such that

$$(\mu^{-1} \partial_t \text{dev}(\boldsymbol{\sigma}), \text{dev}(\boldsymbol{\tau}))_{\Omega} + (\nabla \cdot \boldsymbol{\sigma}, \nabla \cdot \boldsymbol{\tau})_{\Omega} = (\mathbf{F}, \boldsymbol{\tau})_{\Omega} + \langle \mathbf{g}_D, \boldsymbol{\tau} \mathbf{n} \rangle_{\partial\Omega} \quad (7)$$

for any $\boldsymbol{\tau} \in \mathbb{H}_{0,\Gamma_N}(\text{div}, \Omega)$. For further details, the reader is referred to [1].

3.2 PolyDG semi-discrete formulation.

We can now introduce the PolyDG semi-discrete formulation of (7). Let $\mathcal{T}_h$ be a polytopal mesh of the domain Ω , i.e., $\mathcal{T}_h = \bigcup_K \kappa$, κ being a general polygon ($d = 2$). Given a polytopal element κ , we define by $|\kappa|$ its measure and by h_κ its diameter, and set $h = \max_{\kappa \in \mathcal{T}_h} h_\kappa$. We let a polynomial degree $p_\kappa \geq 1$ be associated with each element $\kappa \in \mathcal{T}_h$ and we denote by $p_h : \mathcal{T}_h \rightarrow \mathbb{N}^*$ the piecewise constant function such that $(p_h)|_\kappa = p_\kappa$. Then, we define the discrete space $\mathbf{V}_h = [P_{p_h}(\mathcal{T}_h)]^{d \times d}$, where $P_{p_h}(\mathcal{T}_h) = \Pi_{\kappa \in \mathcal{T}_h} \mathbb{P}_{p_h}(\kappa)$, and $\mathbb{P}_\ell(\kappa)$ is the space of piecewise polynomials in κ of total degree less than or equal to $\ell \geq 1$. We define an interface as the intersection of the $(d - 1)$ -dimensional faces of any two neighboring elements of $\mathcal{T}_h$. We also decompose the set of faces as $\mathcal{F} = \mathcal{F}_h^I \cup \mathcal{F}_h^D \cup \mathcal{F}_h^N$, where $\mathcal{F}_h^I$ contains the internal faces and $\mathcal{F}_h^D$ and $\mathcal{F}_h^N$ the faces of the Dirichlet and Neumann boundary, respectively. We refer the reader to [1] for the main assumptions on $\mathcal{T}_h$. Finally, for sufficiently piecewise smooth vector

and tensor-valued fields $\mathbf{v}$ and $\boldsymbol{\tau}$, respectively, and for any pair of neighbouring elements κ^+ and κ^- sharing a face $F \in \mathcal{F}_h^I$ we introduce the jump and average operators

$$[[\mathbf{v}]] = \mathbf{v}^+ \otimes \mathbf{n}^+ + \mathbf{v}^- \otimes \mathbf{n}^-, \quad [[\boldsymbol{\tau}]] = \boldsymbol{\tau}^+ \mathbf{n}^+ + \boldsymbol{\tau}^- \mathbf{n}^-, \quad \{\{\mathbf{v}\}\} = \frac{\mathbf{v}^+ + \mathbf{v}^-}{2}, \quad \{\{\boldsymbol{\tau}\}\} = \frac{\boldsymbol{\tau}^+ + \boldsymbol{\tau}^-}{2}$$

where $\otimes$ is the tensor product in $\mathbb{R}^d$, $\cdot^\pm$ denotes the trace on F taken within $\kappa^\pm$, and $\mathbf{n}^\pm$ is the outer normal vector to $\partial\kappa^\pm$. Accordingly, on boundary faces $F \in \mathcal{F}_h^D \cup \mathcal{F}_h^N$ we set $[[\mathbf{v}]] = \mathbf{v} \otimes \mathbf{n}$, $[[\boldsymbol{\tau}]] = \boldsymbol{\tau} \mathbf{n}$, and $\{\{\mathbf{v}\}\} = \mathbf{v}$, $\{\{\boldsymbol{\tau}\}\} = \boldsymbol{\tau}$. In the following, we use $\nabla_h \cdot$ to denote the element-wise divergence operator, and we use the short-hand notation $(\cdot, \cdot)_{\mathcal{T}_h} = \sum_{\kappa \in \mathcal{T}_h} \int_\kappa$ and $\langle \cdot, \cdot \rangle_{\mathcal{T}_h} = \sum_{F \in \mathcal{F}_h} \int_F$. We consider the following semi-discrete PolyDG approximation to (??): for any $t \in (0, T]$, find $\boldsymbol{\sigma}_h(t) \in \mathbf{V}_h$ such that

$$\mathcal{M}(\partial_t \boldsymbol{\sigma}_h, \boldsymbol{\tau}_h) + \mathcal{A}(\boldsymbol{\sigma}_h, \boldsymbol{\tau}_h) = \mathbf{F}(\boldsymbol{\tau}_h) \quad \forall \boldsymbol{\tau}_h \in \mathbf{V}_h, \quad (8)$$

$$(\boldsymbol{\sigma}_h(0), \boldsymbol{\tau}_h) = (\boldsymbol{\sigma}_0, \boldsymbol{\tau}_h).$$

where for any $\boldsymbol{\sigma}, \boldsymbol{\tau} \in \mathbf{V}_h$ we have defined

$$\mathcal{M}(\boldsymbol{\sigma}, \boldsymbol{\tau}) = (\mu^{-1} \operatorname{dev}(\boldsymbol{\sigma}), \operatorname{dev}(\boldsymbol{\tau}))_{\mathcal{T}_h}, \quad (9)$$

$$\begin{aligned} \mathcal{A}(\boldsymbol{\sigma}, \boldsymbol{\tau}) &= (\nabla_h \cdot \boldsymbol{\sigma}, \nabla_h \cdot \boldsymbol{\tau})_{\mathcal{T}_h} - \langle \{\{\nabla_h \cdot \boldsymbol{\sigma}\}\}, [[\boldsymbol{\tau}]] \rangle_{\mathcal{F}_h^{I,N}} \\ &\quad - \langle \{\{\nabla_h \cdot \boldsymbol{\tau}\}\}, [[\boldsymbol{\sigma}]] \rangle_{\mathcal{F}_h^{I,N}} + \langle \gamma_e [[\boldsymbol{\sigma}]], [[\boldsymbol{\tau}]] \rangle_{\mathcal{F}_h^{I,N}}, \end{aligned} \quad (10)$$

$$\mathbf{F}(\boldsymbol{\tau}) = (\mathbf{F}, \boldsymbol{\tau})_{\mathcal{T}_h} + \langle \mathbf{g}_D, \boldsymbol{\tau} \mathbf{n} \rangle_{\mathcal{F}_h^D} + \langle \mathbf{g}_N, \gamma_e \boldsymbol{\tau} \mathbf{n} + (\nabla_h \cdot \boldsymbol{\tau}) \mathbf{n} \rangle_{\mathcal{F}_h^N}. \quad (11)$$

Here, $\mathcal{F}_h^{I,N} = \mathcal{F}_h^I \cup \mathcal{F}_h^N$ and the stabilization function $\gamma_e : \mathcal{F}_h^{I,N} \rightarrow \mathbb{R}_+$ is defined as a function of the penalty coefficient $\alpha > 0$ as follows:

$$\gamma_e(\mathbf{x}) = \begin{cases} \alpha \max_{\kappa \in \{\kappa^+, \kappa^-\}} \frac{p_\kappa^2}{h_\kappa}, & \mathbf{x} \in F \in \mathcal{F}_h^I, \\ \alpha \frac{p_\kappa^2}{h_\kappa}, & \mathbf{x} \in F \in \mathcal{F}_h^N. \end{cases}$$

3.3 PolyDG fully-discrete formulation

We introduce a basis $\{\phi_i, i = 1, \dots, N_h\}$ for the space $\mathbf{V}_h$ and express $\boldsymbol{\sigma}_h$ as a linear combination of these basis functions, where the unknown coefficients are functions of time. We collect the latter in the vector $\boldsymbol{\sigma}_h$, denote by M (resp. A) the matrix representation of the bilinear form $\mathcal{M}(\cdot, \cdot)$ (resp. $\mathcal{A}(\cdot, \cdot)$), and by $\mathbf{f}$ the vector representation of the linear functional $\mathbf{F}(\cdot)$. The algebraic formulation of (8) reduces to: for any time $t \in (0, T]$, find $\boldsymbol{\sigma}_h(t) \in \mathbb{R}^{N_h}$ such that

$$M\dot{\boldsymbol{\sigma}}_h(t) + A\boldsymbol{\sigma}_h(t) = \mathbf{f}(t) \quad \forall t \in (0, T], \quad (12)$$

with initial condition $\boldsymbol{\sigma}_h(0) = \boldsymbol{\sigma}_{0,h}$, being the latter the vector representation of the L^2 -projection of $\boldsymbol{\sigma}_0$ onto $\mathbf{V}_h$.

In the following, to integrate in time (12) we use the θ -method: for any $n = 0, \dots, N_T - 1$ find $\boldsymbol{\sigma}_h^{n+1}$ such that

$$(M + \theta\Delta t A)\boldsymbol{\sigma}_h^{n+1} = [M - (1 - \theta)\Delta t A]\boldsymbol{\sigma}_h^n + \Delta t[\theta\mathbf{f}^{n+1} + (1 - \theta)\mathbf{f}^n], \quad (13)$$

with $N_T = T/\Delta t$, $\theta \in [0, 1]$, and where the superscript n means the approximation/evaluation of the given quantity at time $t_n = n\Delta t$, $n = 0, \dots, N_T$. We choose $\theta = \frac{1}{2}$ thus using a second-order Crank–Nicolson scheme. At each time step, we solve the linear system with Flexible AB-GMRES and Kaczmarz inner iterations, as illustrated in algorithm 5. The system matrix has the form $M + \frac{\Delta t}{2}A$, and its conditioning worsens as Δt decreases, being proportional to $\frac{1}{\Delta t}$ (see [2]). In particular, the smaller the time step, the more the matrix inherits the singular behaviour of the mass operator, and the more difficult the linear solve becomes.

Algorithm 5 Crank–Nicolson time integration with flexible AB-GMRES

1: Set $\Delta t = T/N_t$ and $\theta = \frac{1}{2}$.

2: Define

$$\mathbf{M}_L = \mathbf{M} + \theta\Delta t \mathbf{A}, \quad \mathbf{M}_R = \mathbf{M} - (1 - \theta)\Delta t \mathbf{A}.$$

3: Set $\mathbf{U}^0 = \mathbf{u}_0$ and $\mathbf{F}^0 = \mathbf{F}(0)$.

4: **for** $n = 0, 1, \dots, N_t - 1$ **do**

5: Compute $\mathbf{F}^{n+1} = \mathbf{F}(t_{n+1})$ and form

$$\mathbf{b}^{n+1} = \mathbf{M}_R \mathbf{U}^n + \Delta t[\theta\mathbf{F}^{n+1} + (1 - \theta)\mathbf{F}^n].$$

6: Compute the optimal parameters ℓ_{opt} and ω_{opt} with `parameter_tuning`.

7: Solve $\mathbf{M}_L \mathbf{U}^{n+1} = \mathbf{b}^{n+1}$ with `F_AB_GMRES`.

8: Update $\mathbf{U}^n \leftarrow \mathbf{U}^{n+1}$ and $\mathbf{F}^n \leftarrow \mathbf{F}^{n+1}$.

9: **end for**

10: **Output:** $\mathbf{U}^{N_t}$.

4 Numerical Results

In this section, we present the findings of the numerical simulations for the discretized problem introduced in 3.3. The methods were implemented using `lymph`, an open-source `MATLAB` library. We focus on verifying the theoretical convergence of the PolyDG discretization and subsequently analyze the performance of the Flexible AB-GMRES solver preconditioned with Kaczmarz-type inner iterations.

4.1 Theoretical Error Bounds and Expected Observations

Before evaluating the efficiency of the inner-outer solver, it is crucial to outline the expected behavior of the spatial and temporal discretizations as established in the reference literature. For a standard spatial domain $\Omega = (0, 1)^2$ discretized using polytopal meshes (as illustrated in Figure 5), with Dirichlet boundary conditions on the top and right boundaries and Neumann conditions on the remaining edges, the theoretical error analysis of the fully discrete scheme predicts an energy norm error bounded by $\mathcal{O}(\Delta t^2) + \mathcal{O}(h^p)$.

The reference study confirms these theoretical estimates. Log-log plots of the computed error $\|\sigma - \sigma_h\|_E$ demonstrate the expected convergence rates with respect to both the mesh size h for various polynomial degrees ($p = 1, \dots, 4$) and the time step Δt (Figure 6). These established convergence properties serve as the theoretical baseline for how the error should behave, ensuring the structural correctness of the PolyDG formulation before introducing the Kaczmarz-preconditioned GMRES solver dynamics.

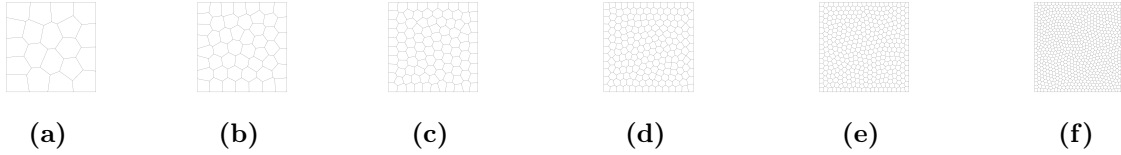


Fig. 5. Considered polytopal meshes: (a) 20 elements, (b) 50 elements, (c) 100 elements ($h \approx 0.1759$), (d) 200 elements ($h \approx 0.1260$), (e) 400 elements ($h \approx 0.0909$), and (f) 800 elements ($h \approx 0.0637$). Boundary edges are highlighted in blue for Dirichlet conditions, and red for Neumann conditions.

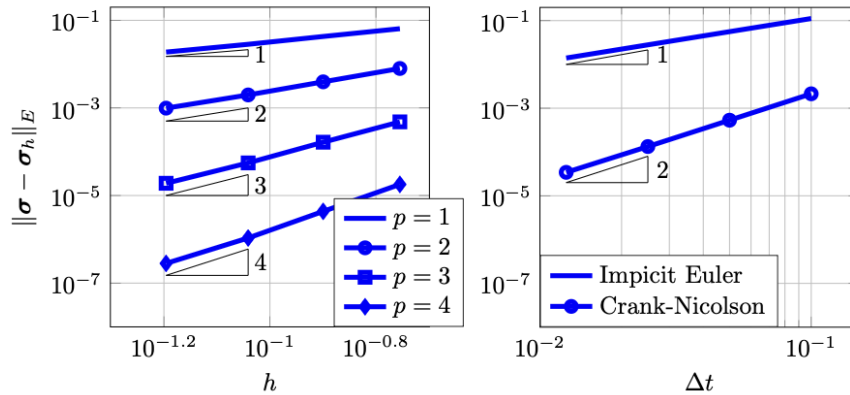


Fig. 6. Left: log-log plot of the expected error $\|\sigma - \sigma_h\|_E$ as a function of the mesh size h for $p = 1, \dots, 4$, by fixing $T = 0.25$ and $\Delta t = 10^{-3}$. Right: log-log plot of the expected error $\|\sigma - \sigma_h\|_E$ as a function of time step Δt fixing the polynomial degree $p = 4$, and $h \approx 0.0909$.

We also set $\mu = 1$. We consider the following exact solution of problem 6:

$$\boldsymbol{\sigma}(\mathbf{x}, t) = \sin(2t) \begin{bmatrix} \sin(\pi x) \sin(\pi y) & 0 \\ 0 & -\sin(\pi x) \sin(\pi y) \end{bmatrix}. \quad (14)$$

$\mathbf{F}(\mathbf{x}, t)$ and the boundary data are computed accordingly.

4.2 Solver Performance and Inner Iteration Stagnation

With the theoretical baseline for the spatial and temporal discretizations established, we now shift our focus to the actual algorithmic performance of the solver.

Test 1: Convergence tests with same time step. The first CN_KACZMARZ (5) test is performed on a mesh of 100 elements (5c) with polynomial degree $p = 1$, total time $T = 0.25$ and time step $dt = 0.025$. The parameters for AB_GMRES are $tol = 10^{-6}$ and $\eta = 10^{-0.5}$. We fixed the outer GMRES iterations to $m_{\max} = 200$ and choose Greedy Randomized Kaczmarz (GRK). At every time step, the parameter tuning function is employed to determine the optimal relaxation parameter ω and the maximum number of inner iterations $\ell_{\max}$, as described in [6].

With $dt = 0.025$ system condition number is $\kappa(A^*) \approx 1.011 \times 10^4$, and the total simulation required 3216.7327 seconds (on average 321.6733 seconds per time step). All useful data from the test is collected in table 1.

This graph is computed with the script *main.m*

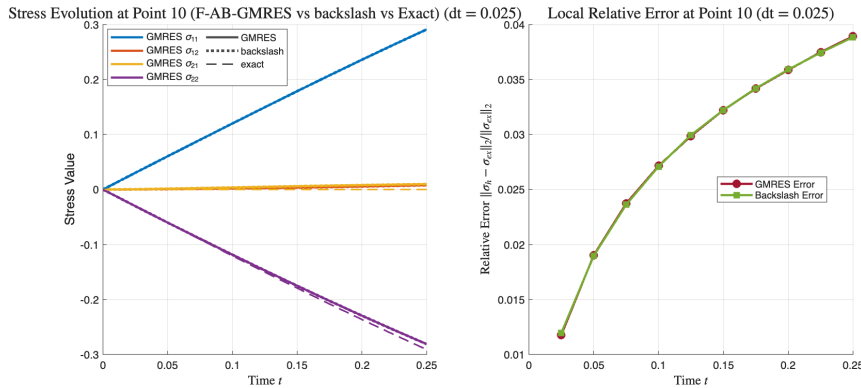


Fig. 7. Left: The evolution of the calculated stress tensor in a random point of the mesh, compared with the default backlash Matlab solver and the exact solution (14). Right: comparison of the computed errors w.r.t. the exact solution at every time step, in a random point of the mesh.

Table 4 reveals a suspicious behavior in the number of inner iterations performed at each outer cycle. First, the parameter-tuning routine ?? consistently reaches the artificial cap of $5m$, where $m = 1200$ is the dimension of the system matrix. This indicates that even a very large number of inner iterations is not sufficient to reduce the residual below the desired threshold. Moreover, the fact that the solver always uses this maximal number of inner iterations at every time step implies that the stopping condition $\|\mathbf{v}_k - AB^{(\ell)}\mathbf{v}_k\|_2 \leq \eta\|\mathbf{v}_k\|_2$, from line 4 of Algorithm 3, is never satisfied. In other words, the relative residual norm never drops below η , even though $\eta = 10^{-0.5}$ is not unreasonably small.

This prompted a further investigation of the inner solver. We analyzed the residual history during the inner iterations and observed that it consistently stagnates without ever dipping below the target η . This behavior is problematic both in the `parameter_tuning` routine (4) and during the actual time-step solves: the inner iterations run up to the maximum allowed number without

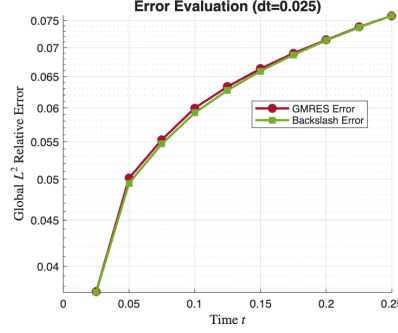


Fig. 8. Global L^2 relative errors on the whole mesh domain, w.r.t. the exact solution.

Table 4. Simulation profile by time step (each time step corresponds to one run of F-AB-GMRES with GRK). The columns report: the time-step index, the current Crank–Nicolson time t , the optimal relaxation parameter ω and the maximum number of inner iterations $\ell_{\max}$ (both obtained by the `parameter_tuning` routine, 4), the number of outer GMRES iterations, the average ratio of inner to outer iterations, and the computational time (in seconds) spent for parameter tuning and for the inner Kaczmarz solves.

Step	Time t	ω	$\ell_{\max}$	Outer It.	Inner It.	Tuning (s)	Inner (s)	Total (s)
1	0.025	0.85	60001	200	60001	9.5486	238.42	248.58
2	0.050	1.00	60001	200	60001	6.7239	1000.10	1008.00
3	0.075	1.05	60001	200	60001	6.4963	229.14	235.89
4	0.100	0.95	60001	200	60001	6.4617	229.69	236.42
5	0.125	1.25	60001	200	60001	6.6374	235.81	242.75
6	0.150	1.10	60001	200	60001	6.5227	240.32	247.30
7	0.175	1.05	60001	200	60001	6.7115	233.37	240.42
8	0.200	1.10	60001	200	60001	6.6037	242.46	249.54
9	0.225	1.00	60001	200	60001	6.8676	237.16	244.31
10	0.250	0.95	60001	200	60001	6.4756	253.12	260.08

converging, consuming most of the computational time (as seen in Table 4) while providing no additional benefit.

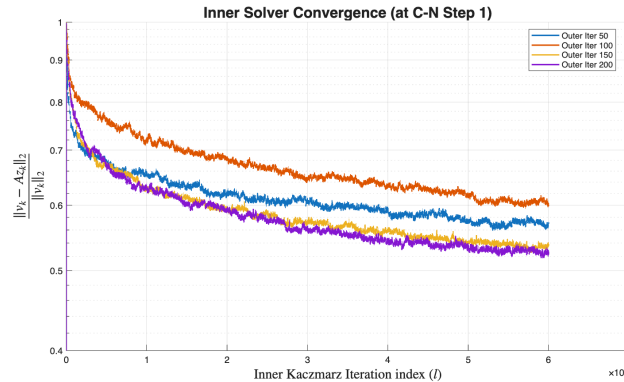


Fig. 9. Inner solver convergence at the first Crank–Nicolson time step. We selected a few outer GMRES iterations and plotted the corresponding residual histories. In all cases, the inner solver reaches the maximum allowed number of iterations without satisfying the target tolerance $\eta = 10^{-0.5}$.

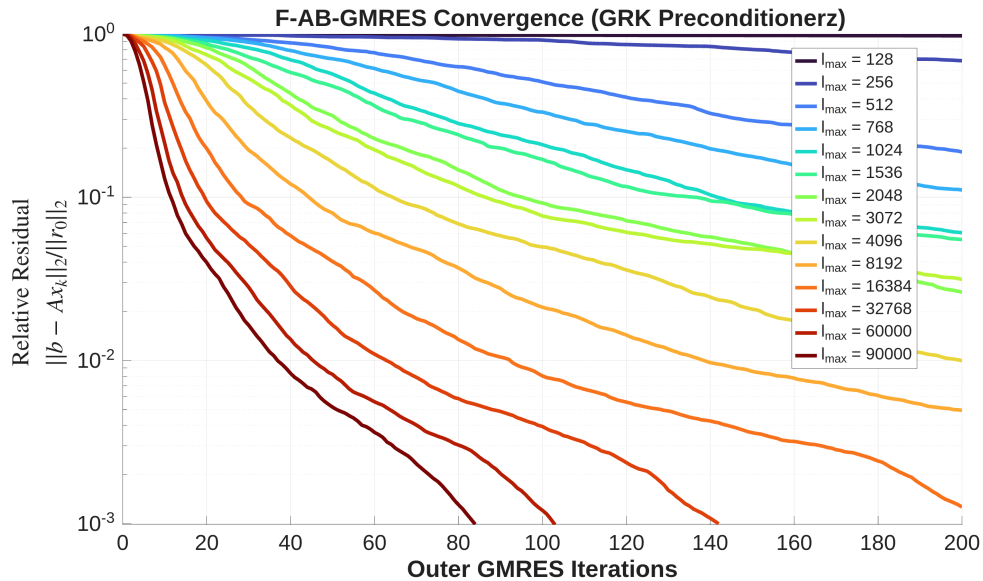
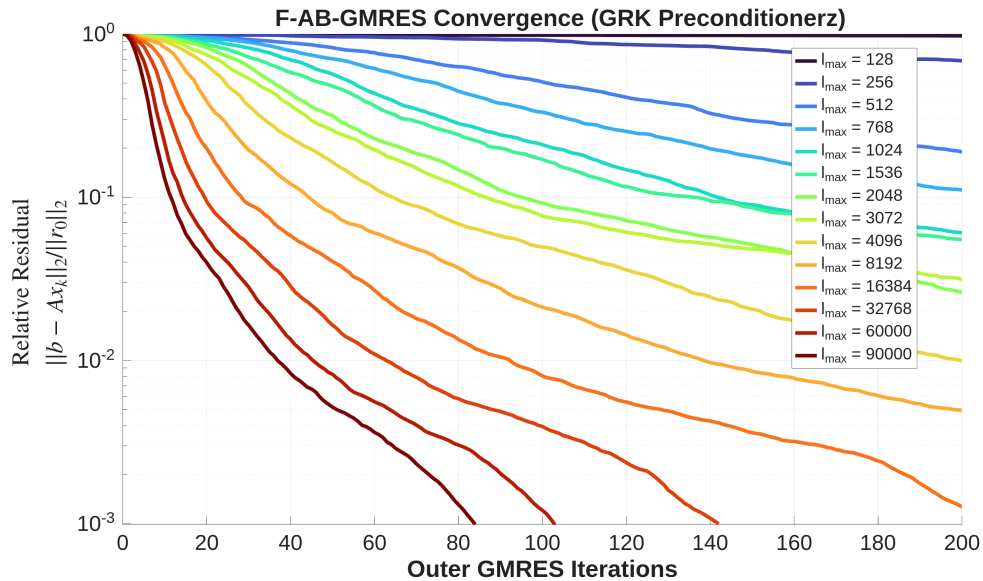
This behaviour might have arisen from the sparse structure of the system matrix A . As seen in picture 10, it shows a clear block pattern. In the Discontinuous Galerkin setting with polynomial degree $p = 1$ on a mesh of 100 elements, each element has exactly 3 local degrees of freedom. This appears in the matrix as a sequence of nearly uncoupled 3×3 diagonal blocks, together with smaller off-diagonal entries that represent the interface fluxes coupling neighboring elements.

A proposed solution could be to use a Block Kaczmarz method. Instead of projecting onto a single row, we could project onto a subspace defined by a block of rows (in this case blocks of 3 rows). This might better respect the local algebraic coupling and improve convergence.

In this work, we did not replace the point-wise Kaczmarz projection with a block method. Instead we introduced an adaptive strategy: the idea is to adjust the number of inner iterations ℓ_k at each outer GMRES step, based on the behavior of the relative residual. If the residual is still decreasing, more inner iterations are allowed. If the residual is flat, the inner solver is stopped early. This prevents the solver from wasting thousands of iterations on steps where the Kaczmarz sweep has already stagnated, and it improves the overall runtime of the simulation. This method is presented in section 4.3.

Note: An interesting observation derived from our numerical tests concerns the maximum number of inner iterations. If we use too few inner iterations —such as 128 or 256 in our extreme cases— for a poorly conditioned matrix the method stagnates. This happens because the inner Kaczmarz solver acts as a preconditioner; with an insufficient number of iterations, it fails to adequately approximate the action of the inverse matrix. Consequently, the search directions fed to the outer F-AB-GMRES solver are too poor to effectively minimize the residual. Thus, even as the outer iterations increase and the Krylov subspace expands, the solver makes no meaningful progress. This was specifically observed for the matrix derived from 100-element mesh with $p = 1$ and $dt = 0.05$, with $k(A) = 1.1684 \times 10^4$.

This graph is computed with the script *check_stagnation.m*



Test 2: Convergence tests with varying time step. For our next kind of tests, we repeated the same simulation for several values of Δt . The Crank–Nicolson scheme is expected to show a global error of order $\mathcal{O}(\Delta t^2) + \mathcal{O}(h^p)$, where h is the mesh size and p is the polynomial degree. In our setting $p = 1$ and the mesh is fixed, so the spatial error is constant and the time step is the only parameter that changes.

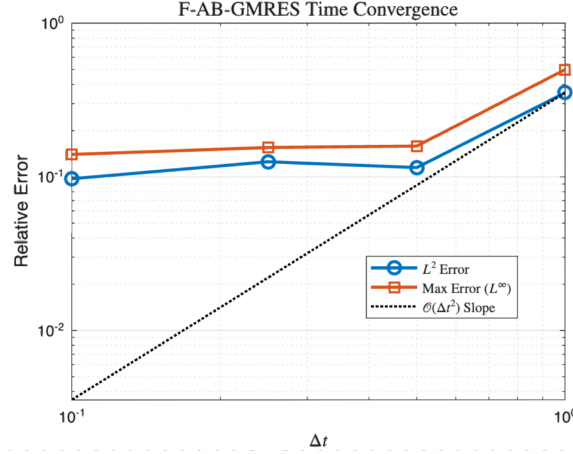


Fig. 11. Results for a 100-element mesh with $p = 1$. The error follows the Δt^2 slope, then settles on the h^1 since the mesh is not refined enough for the error to dip lower.

Table 5. Test on a mesh of 10 elements, with $p = 1$. Note the increasing conditioning number (see Remark 3) of the system matrix as $\Delta t \rightarrow 0$ and the variation of the number of inner and outer iterations. The L^2 error stays almost unchanged.

Δt	Cond.	Avg. Outer	Avg. Inner/Outer	ω_{opt} (Step 1)	ℓ_{max} (Step 1)	Avg. Step Time (s)	Total Time (s)	L^2 Error
0.025	1133.4	27	5551	1.40	5239	1.1689	11.944	0.27238
0.0025	1111.2	73	132	1.10	107	0.08457	8.9148	0.27886
0.00025	3655.1	120	28	1.05	26	0.05982	64.321	0.28032

Table 6. Test on a mesh of 50 elements with $p = 1$. As Δt decreases, the conditioning number of the system matrix slightly decreases (see Remark 3). The L^2 error again remains almost unchanged.

Δt	Cond.	Avg. Outer	Avg. Inner/Outer	ω_{opt} (Step 1)	ℓ_{max} (Step 1)	Avg. Step Time (s)	Total Time (s)	L^2 Error
0.025	5297.6	112	30001	1.20	30001	44.901	449.53	0.091014
0.0125	4837.7	78	30001	1.35	30001	31.89	638.78	0.09177
0.00833	4673.8	67	29987	1.45	30001	27.713	832.84	0.092009
0.00625	4593.9	74	19659	1.30	18539	20.79	833.36	0.092122

Remark 3. The conditioning number decreases for the 100p1 tests because these time steps are still too large compared to the spatial mesh (100 elements). The term $\frac{\Delta t}{2}A$ is still large enough to dominate the system. Refining Δt even more would require too much computational effort.

On the other hand, in the 10p1 case with $\Delta t = 0.00025$, the term $\frac{\Delta t}{2}A$ is small enough for the system to inherit the singular behavior of M , which causes the conditioning number to increase.

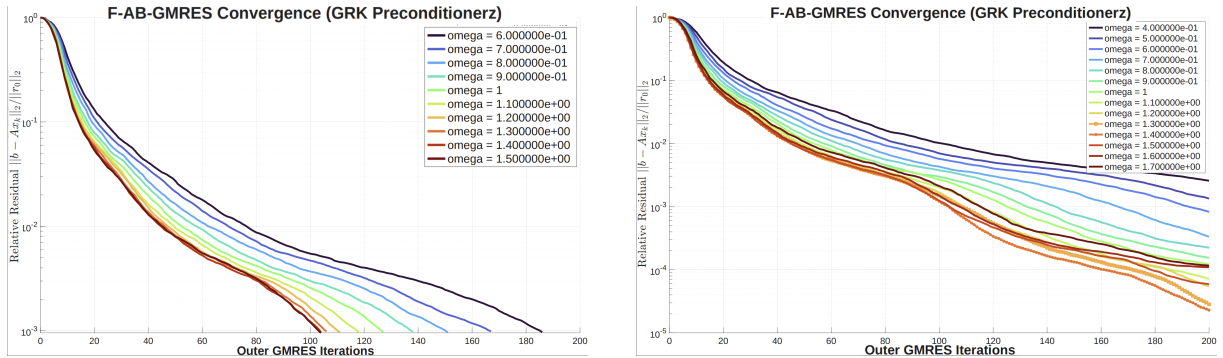
4.3 Adatptive AB-GMRES method

As observed in Section 4.2 , the standard formulation of the F-AB-GMRES method is prone to inner iteration stagnation. The Kaczmarz-type inner solver frequently reaches the maximum allowed number of iterations without satisfying the target tolerance η . This results in a massive waste of computational time, as the solver performs thousands of unproductive inner sweeps that fail to significantly improve the preconditioning vector z_k .

To mitigate this, we introduce an adaptive F-AB-GMRES strategy. Rather than forcing the inner solver to run up to a static $l_{\max}$, the adaptive approach monitors the relative residual norm during the inner loop. If the residual fails to decrease by a defined threshold (stagnation), it has done too few inner iterations therefore the maximum number of inner iterations for the subsequent outer step is then multiplied by a growth factor. This prevents the solver from wasting iterations when the Kaczmarz sweep has flattened out, dynamically shifting the computational effort back to the outer GMRES solver.

Relaxation Parameter Tuning The effectiveness of the Kaczmarz inner solver is highly sensitive to the relaxation parameter ω . The figures above demonstrate the parameter tuning process, highlighting the distinct optimal ω values required to minimize the residual norm for outer tolerances of 10^{-3} and 10^{-6} , respectively.

This graph is computed with the script *check stagnation.m*



Initializing the Adaptive Strategy and Growth Factor Analysis A critical hyperparameter in the adaptive scheme is the initial cap on inner iterations. We tested multiplying the current limit by a factor of 1.5 versus a factor of 2.

Our numerical experiments evaluated several starting thresholds. The results demonstrate that initializing the adaptive method with a maximum of 256 inner iterations provides the optimal balance. It allows the Kaczmarz method to perform enough initial work to form a useful search direction without falling into early stagnation plateaus. The data indicates that a step increase of $\times 2$ outperforms $\times 1.5$. The more aggressive scaling allows the inner solver to break through stagnation plateaus more decisively, reducing the total wall time of the simulation.

Table 7. Adaptive F-AB-GMRES Performance: Growth Factor Comparison. Tests performed with $\Delta t = 0.05$, $\eta = 10^{-0.5}$, $tol = 0.001$, stagnation tolerance = 0.3, max outer iteration = 200, and a baseline Max Error (backslash) of 4.5699×10^{-2} . In green we highlighted the fastest one. This table is computed with the script *error single dt different inner.m*

Initial l_{max}	Growth Factor	Restart Cycles	Final l_{max}	Outer Iters	Total Inner Iters	Avg Inner/Outer	Total Time (s)	Max Error
16	x1.5	14	4710	3000	2803019	9.343397e+02	166.5246	4.812955e-02
	x2	10	16384	2400	6617349	2.757229e+03	291.6714	4.652183e-02
32	x1.5	13	6245	2800	3713660	1.326307e+03	193.4789	4.796096e-02
	x2	8	8192	2000	3357762	1.678881e+03	155.2777	4.543435e-02
48	x1.5	13	9368	2800	5565331	1.987618e+03	250.5805	4.962765e-02
	x2	8	12288	2000	4960162	2.480081e+03	220.6051	4.641072e-02
64	x1.5	11	5540	2400	3280620	1.366925e+03	167.8536	4.607380e-02
	x2	7	8192	2000	3754835	1.877418e+03	169.3376	4.934817e-02
128	x1.5	10	7383	2200	4355210	1.979641e+03	236.2738	4.709308e-02
	x2	5	4096	1600	2522902	1.576814e+03	114.2263	4.630888e-02
192	x1.5	9	7383	2000	4328355	2.164177e+03	229.6109	4.797348e-02
	x2	6	12288	1600	4924342	3.077714e+03	215.2013	4.986668e-02
256	x1.5	8	6561	2000	3931650	1.965825e+03	212.6820	4.655442e-02
	x2	4	4096	1600	3310512	2.069070e+03	142.7157	4.618091e-02
384	x1.5	7	6561	1800	3879493	2.155274e+03	199.2166	4.587829e-02
	x2	5	12288	1600	5577462	3.485914e+03	242.9669	4.969356e-02
512	x1.5	7	8748	1800	5115878	2.842154e+03	264.9676	4.881651e-02
	x2	4	8192	1400	3666930	2.619236e+03	158.9553	4.743328e-02
768	x1.5	6	8748	1600	5066105	3.166316e+03	220.2701	4.735928e-02
	x2	4	12288	1400	5495551	3.925394e+03	245.1672	4.743294e-02
1024	x1.5	5	7776	1400	4437069	3.169335e+03	192.5834	4.805502e-02
	x2	4	16384	1400	7320606	5.229004e+03	312.3353	4.773349e-02
2048	x1.5	4	10368	1200	5775411	4.812842e+03	286.8904	4.996186e-02
	x2	1	4096	800	2443947	3.054934e+03	108.2359	4.613976e-02
4096	x1.5	1	6144	800	4072021	5.090026e+03	204.1800	4.651976e-02
	x2	1	8192	600	3256739	5.427898e+03	148.9185	4.585118e-02
8192	x1.5	0	8192	600	4883442	8.139070e+03	242.6376	4.893527e-02
	x2	0	8192	600	4885276	8.142127e+03	208.3079	4.567295e-02
15000	x1.5	0	15000	400	5971504	1.492876e+04	299.5882	4.564155e-02
	x2	0	15000	400	5969217	1.492304e+04	239.3149	4.565112e-02
30000	x1.5	0	30000	157	4681543	2.981875e+04	219.6224	4.572642e-02
	x2	0	30000	156	4651737	2.981883e+04	198.5619	4.577390e-02
60000	x1.5	0	60000	104	6188033	5.950032e+04	304.1641	4.572867e-02
	x2	0	60000	105	6248411	5.950868e+04	274.2871	4.574476e-02

Performance Comparison Adaptive vs. Non-Adaptive To properly assess the efficiency of the adaptive strategy, we compare its performance against the standard, non-adaptive F-AB-GMRES method.

Initially, we observe strange behavior, which is, however, explained by this observation that had not immediately caught our eye. The standard F-AB-GMRES method artificially halts once it hits the fixed budget of 200 iterations, leaving the system unconverged. In contrast, the adaptive scheme acts as a restarted GMRES: it evaluates the true external residual and repeatedly re-triggers the solver until the 10^{-6} tolerance is strictly satisfied. The reported 2600 outer iterations must be seen as the cumulative sum of 13 consecutive restart cycles (13×200 iterations) required to successfully force convergence.

To resolve this discrepancy and establish a uniform baseline, we relaxed the convergence tolerance to 10^{-3} . This threshold allows both methods to successfully converge within the fixed iteration budget, facilitating a direct comparison of their computational efficiency.

Table 8. Overall Performance Summary: Adaptive vs. Standard F-AB-GMRES ($N_t = 5$, $\Delta t = 0.05$). These two tables are computed with the script *error with dt that varies.m*

Metric	Adaptive F-AB-GMRES	Standard (No Adaptive)
Initial l_{max}	2048 (Growth: x2)	60000 (Static)
Total Outer Iters	3600	413
Total Inner Iters	12224948	24515059
Total CPU Time (s)	719.93	1014.75
Avg. Time / Step (s)	122.22	200.22
Max Error	7.581016e-02	7.614793e-02

Table 9. Step-by-Step Breakdown: Adaptive vs. Standard F-AB-GMRES Iteration Counts

Time Step	Adaptive F-AB-GMRES		Standard (No Adaptive)
	Outer / Inner Iters	Final l_{max}	Outer / Inner Iters
Step 1 ($t = 0.05$)	1000 / 4075785	8192	105 / 6249308
Step 2 ($t = 0.10$)	600 / 1629670	4096	92 / 5464887
Step 3 ($t = 0.15$)	800 / 3258525	8192	79 / 4688385
Step 4 ($t = 0.20$)	600 / 1630303	4096	73 / 4325603
Step 5 ($t = 0.25$)	600 / 1630665	4096	64 / 3786876

5 Conclusion

Here we wrap up the main conclusions of this work.

On our benchmark tests, greedy Kaczmarz methods (GK, GRK) outperform NE-SOR and classical Kaczmarz in terms of outer iterations and total CPU time. Their advantage is especially visible for ill-conditioned systems and tighter inner tolerances.

At the same time, the performance of these methods is very sensitive to the parameters w and ℓ_{opt} so they must always be tuned accordingly.

Despite the promising baseline performance of our implementations, we observe that, when applied to PolyDG matrices, the results remain correct and consistent with the theory but require much longer computation times. This slowdown comes from row-wise Kaczmarz updates failing to capture the characteristic block structure of the system, which causes the inner residual to stagnate rapidly and forces an excessive number of inner iterations.

We then implemented an adaptive F-AB-GMRES method designed to avoid these wasted iterations. This strategy only partially mitigates this problem: in a baseline test with $N_t = 5$, $\Delta t = 0.05$ and tolerance 10^{-3} , the adaptive version reduced the total CPU time to 719.93 seconds from the original 1014.75, and cut the total number of inner iterations from 24 515 059 to 12 224 948.

From these results, we conclude that Kaczmarz-type inner preconditioners are a valid and sometimes advantageous alternative to NE-SOR on standard systems, but their direct application to PolyDG matrices is not yet scalable and needs to be handled with attention.

A natural and promising extension of our work would be to replace the row-wise Kaczmarz inner solver with a Block Kaczmarz method that, instead of projecting row by row, works on full blocks, exploiting the matrix structure. If implemented and integrated into the existing adaptive F-AB-GMRES framework this extension could perhaps enable our solver to tackle large-scale, high-order space–time PolyDG systems efficiently.

References

- [1] P. F. Antonietti, M. Botti, A. Cancrini, and I. Mazzieri. A polytopal discontinuous galerkin method for the pseudo-stress formulation of the unsteady stokes problem. *Computer Methods in Applied Mechanics and Engineering*, 447:118404, 2025. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2025.118404>. URL <https://www.sciencedirect.com/science/article/pii/S0045782525006760>.
- [2] P. F. Antonietti, A. Cancrini, and G. Ciaramella. Numerical verification of polydg algebraic solvers for the pseudo-stress stokes problem, 2026. URL <https://arxiv.org/abs/2512.02537>.
- [3] Z.-Z. Bai and W.-T. Wu. On greedy randomized kaczmarz method for solving large sparse linear systems. *SIAM Journal on Scientific Computing*, 40(1):A592–A606, 2018. doi: 10.1137/17M1137747. URL <https://doi.org/10.1137/17M1137747>.
- [4] A. Bjorck and T. Elfving. Accelerated projection methods for computing pseudoinverse solutions of systems of linear equations. *BIT*, 19:145–163, 06 1979. doi: 10.1007/BF01930845.
- [5] G. Ciaramella and M. J. Gander. *Iterative Methods and Preconditioners for Systems of Linear Equations*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2022. doi: 10.1137/1.9781611976908. URL <https://epubs.siam.org/doi/abs/10.1137/1.9781611976908>.
- [6] Y.-S. Du, K. Hayami, N. Zheng, K. Morikuni, and J.-F. Yin. Kaczmarz-type inner-iteration preconditioned flexible gmres methods for consistent linear systems, 2021. URL <https://arxiv.org/abs/2006.10818>.
- [7] K. Hayami, J.-F. Yin, and T. Ito. Gmres methods for least squares problems. *SIAM Journal on Matrix Analysis and Applications*, 31(5):2400–2430, 2010. doi: 10.1137/070696313. URL <https://doi.org/10.1137/070696313>.
- [8] Y. Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, second edition, 2003. doi: 10.1137/1.9780898718003. URL <https://epubs.siam.org/doi/abs/10.1137/1.9780898718003>.